Assignment 4 - Data Wrangling

In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (10, 5)
print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

In [4]:
```
# 1. Load the Bangalore Housing dataset
df = pd.read_csv('Bengaluru_House_Data.csv')

print(f"Original Dataset Shape: {df.shape}")
print("Original Columns:", df.columns.tolist())

# 2. Clean column names: strip whitespace, lowercase, replace spaces with underscores
df.columns = (
    df.columns.str.strip()
              .str.lower()
              .str.replace(' ', '_')
              .str.replace(r'[^\w\s]', '', regex=True)
)

print("\nCleaned Columns:", df.columns.tolist())
df.head(3)
```

```
Original Dataset Shape: (13320, 9)
Original Columns: ['area_type', 'availability', 'location', 'size', 'society', 'total_sqft', 'bath', 'balco
ny', 'price']

Cleaned Columns: ['area_type', 'availability', 'location', 'size', 'society', 'total_sqft', 'bath', 'balcon
y', 'price']
```

Out[4]:

	area_type	availability	location	size	society	total_sqft	bath	balcony	price
0	Super built-up Area	19-Dec	Electronic City Phase II	2 BHK	Coomee	1056	2.0	1.0	39.07
1	Plot Area	Ready To Move	Chikka Tirupathi	4 Bedroom	Theanmp	2600	5.0	3.0	120.00
2	Built-up Area	Ready To Move	Uttarahalli	3 BHK	NaN	1440	2.0	3.0	62.00

In [5]:
```
# Check missing value counts
print("Missing values per column:\n", df.isnull().sum())

# 1. Drop 'society' (too many missing values) and 'availability' (not needed for basic pricing)
df = df.drop(columns=['society', 'availability'], errors='ignore')

# 2. Fill missing categorical values
df['location'] = df['location'].fillna('Unknown')
df['size'] = df['size'].fillna(df['size'].mode()[0])

# 3. Fill missing numerical values with median
df['bath'] = df['bath'].fillna(df['bath'].median())
df['balcony'] = df['balcony'].fillna(df['balcony'].median())

# 4. Standardize 'size' to extract numeric BHK count
df['bhk'] = df['size'].apply(lambda x: int(str(x).split(' ')[0]) if pd.notnull(x) else np.nan)
df.drop(columns=['size'], inplace=True)

# 5. Clean 'total_sqft' (handles ranges like '2100 - 2850' by taking average, drops unit strings)
def convert_sqft_to_num(x):
    tokens = str(x).split('-')
    if len(tokens) == 2:
        return (float(tokens[0]) + float(tokens[1])) / 2
    try:
        return float(x)
    except:
        return None

df['total_sqft'] = df['total_sqft'].apply(convert_sqft_to_num)
```

```
df = df.dropna(subset=['total_sqft'])

# Derive price per square foot (Price is in Lakhs, so multiply by 100,000)
df['price_per_sqft'] = (df['price'] * 100000) / df['total_sqft']

print(f"\nRemaining missing values: {df.isnull().sum().sum()}")
df.head(3)
```

```
Missing values per column:
 area_type          0
availability       0
location           1
size              16
society         5502
total_sqft         0
bath              73
balcony          609
price              0
dtype: int64

Remaining missing values: 0
```

Out[5]:

	area_type	location	total_sqft	bath	balcony	price	bhk	price_per_sqft
0	Super built-up Area	Electronic City Phase II	1056.0	2.0	1.0	39.07	2	3699.810606
1	Plot Area	Chikka Tirupathi	2600.0	5.0	3.0	120.00	4	4615.384615
2	Built-up Area	Uttarahalli	1440.0	2.0	3.0	62.00	3	4305.555556

In [6]:

```
# Create supplementary data for major Bangalore localities (Tier classification)
top_localities = df['location'].value_counts().head(10).index.tolist()

supplementary_tier_df = pd.DataFrame({
    'location': top_localities,
    'region_tier': ['Tier 1' if i < 5 else 'Tier 2' for i in range(len(top_localities))],
    'safety_index': [8.5 - (i * 0.2) for i in range(len(top_localities))]
})
```

```
# Left join supplementary demographics on location
df = pd.merge(df, supplementary_tier_df, on='location', how='left')

# Fill non-top localities with default values
df['region_tier'] = df['region_tier'].fillna('Other')
df['safety_index'] = df['safety_index'].fillna(df['safety_index'].mean())

print("Merged supplementary data successfully!")
display(df[['location', 'area_type', 'price', 'region_tier', 'safety_index']].head(3))
```

```
Merged supplementary data successfully!
```

	location	area_type	price	region_tier	safety_index
0	Electronic City Phase II	Super built-up Area	39.07	Other	7.816885
1	Chikka Tirupathi	Plot Area	120.00	Other	7.816885
2	Uttarahalli	Built-up Area	62.00	Tier 2	7.300000

In [7]:

```
# Filter realistic properties:
# 1. Properties where sqft per BHK is at least 300 sqft
# 2. BHK count between 1 and 6
# 3. Bathrooms not exceeding BHK + 2
subset_df = df[
    (df['total_sqft'] / df['bhk'] >= 300) &
    (df['bhk'].between(1, 6)) &
    (df['bath'] <= df['bhk'] + 2)
].copy()

print(f"Original record count: {len(df)}")
print(f"Filtered subset record count: {len(subset_df)}")
```

```
Original record count: 13274
Filtered subset record count: 12431
```

In [8]:

```
# Prepare categorical columns for modeling: 'area_type' and 'region_tier'
# Use One-Hot Encoding with pd.get_dummies
df_encoded = pd.get_dummies(
```

```
    subset_df,
    columns=['area_type', 'region_tier'],
    drop_first=True
)

print("Columns after encoding:")
print(df_encoded.columns.tolist()[:10], "... [truncated]")
df_encoded.head(3)
```

```
Columns after encoding:
['location', 'total_sqft', 'bath', 'balcony', 'price', 'bhk', 'price_per_sqft', 'safety_index', 'area_type_
Carpet  Area', 'area_type_Plot  Area'] ... [truncated]
```

Out[8]:

	location	total_sqft	bath	balcony	price	bhk	price_per_sqft	safety_index	area_type_Carpet Area	area_type_Plot Area	are
0	Electronic City Phase II	1056.0	2.0	1.0	39.07	2	3699.810606	7.816885	False	False	
1	Chikka Tirupathi	2600.0	5.0	3.0	120.00	4	4615.384615	7.816885	False	True	
2	Uttarahalli	1440.0	2.0	3.0	62.00	3	4305.555556	7.300000	False	False	

In [9]:

```
# Aggregate average price (in Lakhs) and average size by area_type and bhk
print("=== Price & Size Summary by Area Type ===")
area_summary = subset_df.groupby('area_type').agg(
    total_properties=('price', 'count'),
    avg_price_lakhs=('price', 'mean'),
    median_price_lakhs=('price', 'median'),
    avg_sqft=('total_sqft', 'mean')
).reset_index()
display(area_summary)

print("\n=== Average Price by BHK ===")
bhk_summary = subset_df.groupby('bhk')['price'].agg(['count', 'mean', 'min', 'max']).reset_index()
display(bhk_summary)
```

```
=== Price & Size Summary by Area Type ===
```

	area_type	total_properties	avg_price_lakhs	median_price_lakhs	avg_sqft
0	Built-up Area	2298	103.095977	68.000	1602.800183
1	Carpet Area	80	84.858812	68.545	1271.142063
2	Plot Area	1348	230.372559	150.000	2187.717055
3	Super built-up Area	8705	91.897991	65.000	1468.599076

```
=== Average Price by BHK ===
```

	bhk	count	mean	min	max
0	1	642	44.213598	9.00	750.0
1	2	5493	59.855851	8.44	650.0
2	3	4748	111.403876	12.00	2000.0
3	4	1256	289.576879	25.00	2912.0
4	5	202	376.202970	44.50	2736.0
5	6	90	315.522222	58.00	2800.0

In [10]:

```
# Visualize price before outlier removal
plt.figure(figsize=(10, 3))
sns.boxplot(x=subset_df['price'], color='salmon')
plt.title('House Price Distribution (Before Capping Outliers)')
plt.xlabel('Price (in Lakhs)')
plt.show()

# Detect and cap outliers using IQR on 'price'
Q1 = subset_df['price'].quantile(0.25)
Q3 = subset_df['price'].quantile(0.75)
IQR = Q3 - Q1

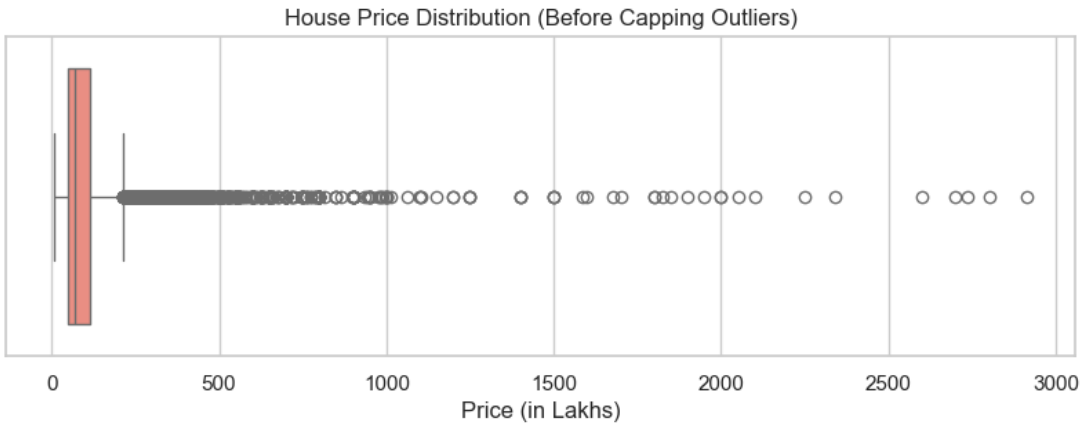
lower_limit = Q1 - 1.5 * IQR
```

```
upper_limit = Q3 + 1.5 * IQR

print(f"Lower Bound: {lower_limit:.2f} Lakhs")
print(f"Upper Bound: {upper_limit:.2f} Lakhs")

# Cap extreme values to upper and lower limits
subset_df['price_cleaned'] = np.clip(subset_df['price'], lower_limit, upper_limit)

# Visualize price after outlier capping
plt.figure(figsize=(10, 3))
sns.boxplot(x=subset_df['price_cleaned'], color='mediumseagreen')
plt.title('House Price Distribution (After IQR Capping)')
plt.xlabel('Price (in Lakhs)')
plt.show()
```




```
Lower Bound: -50.00 Lakhs
Upper Bound: 214.00 Lakhs
```

